Spec-to-Code Platform — Production Technology Stack

Status key: **Live** = built and verified in this project · **Planned** = decided, not yet built · **Evaluating** = discussed, no final decision · **Deferred** = considered and explicitly set aside

Application Layer

Tool	Role	Status
Python 3.12	Runtime	Live
FastAPI	Web framework, all API endpoints	Live
Uvicorn	ASGI server	Live
Pydantic	Request/response validation, structured output schemas	Live
SQLAlchemy	ORM, all database models	Live
python-dotenv	Environment variable loading (`.env`)	Live — **planned to move to Key Vault**, see Secrets below

AI / LLM Layer

Tool	Role	Status
Claude (Anthropic API) — `claude-sonnet-5`	Core generation, review, understanding, all reasoning agents	Live
Claude — `claude-opus-5`	Higher-capability tier, available via `claude_model.py`	Live (available, not yet routed to by default)
`anthropic` Python SDK, raw, with forced `tool_choice`	Structured output for 7 of 8 agent modules	Live

Tool	Role	Status
Agno (`agno.agent.Agent`, `agno.models.anthropic.Claude`)	Structured output via Pydantic `output_schema`, replacing raw SDK calls	Live for 1 of 8 agent modules (`spec_extraction_agent.py`); rest gated on a real-key streaming-reliability test (Sprint 2)

Orchestration

Tool	Role	Status
LangGraph	Pipeline orchestration (Tier 1 rules → Tier 3 ML → Tier 4 fine-tuned → Tier 5 Claude → Guardrails)	Live

Data Layer

Tool	Role	Status
PostgreSQL 16	Production database — operational tables, audit trail, training corpus, model registry, observability	Live, verified end-to-end against a real instance
SQLite	Local development default	Live (dev only — being phased out of any real deployment)
`psycopg2`	PostgreSQL driver	Live
pgvector	Vector similarity search for RAG retrieval, replacing TF-IDF	Planned (Sprint 4)
Alembic	Schema migrations	Planned (Sprint 3) — currently "delete the `.db` file" is the real answer to a schema change, flagged repeatedly as a gap

ML Layer

Tool	Role	Status
XGBoost	Tier 3 classifier	Live, held in shadow mode — blocked on training data volume, not a code issue

Tool	Role	Status
scikit-learn	Supporting ML utilities	Live
pandas	Data manipulation	Live
MLflow	Experiment tracking / model versioning	**Deferred** — a Postgres-backed `model_registry` table with explicit shadow → candidate → active → retired promotion was built instead, to avoid standing up a separate service for this project's actual scale

Frontend

Tool	Role	Status
Vanilla HTML / CSS / JavaScript	The actual, live, maintained frontend	Live
Next.js	Parallel frontend	Built early in the project, has lagged in features since (e.g. never got the metadata upload panel) — **retire-or-catch-up decision pending** (Sprint 5)

Testing

Tool	Role	Status
Playwright	End-to-end browser testing — used to verify nearly every feature built this project	Live, used extensively in development; **not yet wired into CI**
pytest	Unit/integration testing	Minimal — one saved test file exists despite dozens of manually-verified features; real coverage is Sprint 3's top priority

Cloud Platform — Azure

Tool	Role	Status
Azure Database for PostgreSQL (Flexible Server)	Managed production database	Planned (Sprint 1) — schema validated to apply cleanly; `sslmode=require` confirmed as the one required connection-string change
Azure App Service **or** Azure Container Apps	Application hosting	Planned (Sprint 2) — decision not yet finalized; Container Apps favored for scale-to-zero given bursty (LLM-call-driven, not constant) traffic
Azure Key Vault	Secrets management (`ANTHROPIC_API_KEY`, `DATABASE_URL`)	Planned (Sprint 1)
Microsoft Entra ID	Authentication — via App Service Easy Auth, and separately available for direct Postgres token auth	Planned (Sprint 2) — no authentication exists today; this is the top-priority production gap
Application Insights	Frontend + backend observability, alerting	Planned (Sprint 4) — complements, doesn't replace, the in-database `request_log` table already built
ADLS Gen2 / Blob Storage	Object storage	Discussed as a Delta Lake backing store

Analytical Data Layer (evaluated, not built)

Tool	Role	Status
Delta Lake, via `delta-rs` (no Spark/Databricks)	Analytical/historical copy of operational data, time-travel for audit reconstruction	**Evaluating** — architecture discussed in detail (Postgres stays OLTP, Delta Lake as OLAP fed by batch ETL), explicitly chosen as standalone rather than bridging to an existing Databricks pipeline, but no code has been built yet
DuckDB	Ad-hoc querying of Delta tables without Spark	Evaluating, same status as above

CI/CD (planned, not yet built)

Tool	Role	Status
GitHub Actions or Azure DevOps	Automated build/test/deploy pipeline	Planned (Sprint 1 for basic checks, Sprint 6 for full pipeline with staging slots) — the direct, repeated root cause of this project's real production incidents (stale files, missing modules) has been manual, un-validated file copying

Document/Artifact Generation (build-time tooling, not part of the running app)

Tool	Role
openpyxl	Project plan and sprint plan workbook generation
nbformat	One-off Jupyter notebook generation (superseded by plain-text spec format per actual usage)
LibreOffice (`soffice`)	Spreadsheet formula recalculation and rendering for verification

Notable decisions worth remembering the reasoning for

Postgres over a separate vector DB: pgvector runs as a Postgres extension, avoiding a second database technology to operate and secure.
A hand-rolled model registry over MLflow: at this project's real scale, a Postgres table with an explicit promotion workflow does the job MLflow would, without adding a service.
Delta Lake kept standalone rather than bridged to the existing Databricks CDISC pipeline: a deliberate choice, not an oversight — revisit only if that other pipeline becomes a real integration target.
Agno migration is incremental, not big-bang: one real, documented Agno reliability issue (structured output can silently force non-streaming, failing on long calls) means each agent gets migrated and verified individually, not all at once.